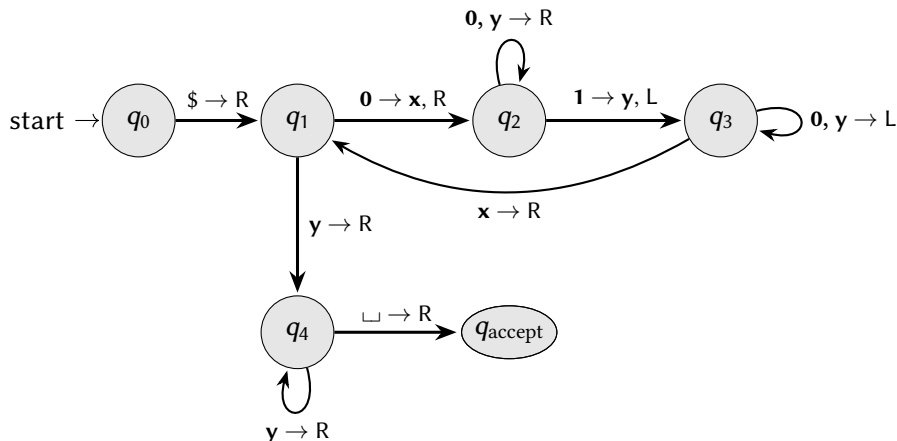


Turing Machines, Continued

Let's keep talking about the TMs that we introduced last week. Specifically, recall that a TM is an automaton in which we can overwrite characters, and in which we can move either left or right on the input string:



Can we do Better?

TMs are good for describing algorithms because they can be “programmed” like computers.

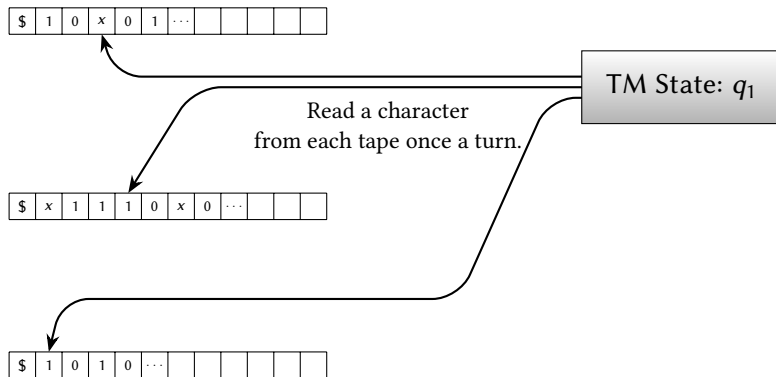
But is it possible to make even more powerful automata?

A (non-exhaustive) list of possible improvements:

- Use multiple tapes.
- Use non-determinism.
- Use RAM instead of linear memory.
- ...

Review: Multi-tape Turing Machines

Like regular TMs, but with multiple tapes:



Review: Emulating Multitape TMs

Idea: flatten the three tapes into one:

$$\begin{array}{l} \$10q_1x01\sqcup \\ \$x11q_110x0\sqcup \\ \$q_11010\sqcup \end{array}$$

into

$$\#\$10q_1x01\#\$x11q_110x0\#\$q_11010\sqcup$$

Notice: The tape head is in a different position on every string. In this construction the state is actually written on the tape — that way we can move the real tape head around without forgetting where we are on this particular tape.

Review: Emulating Multitape TMs: A single step

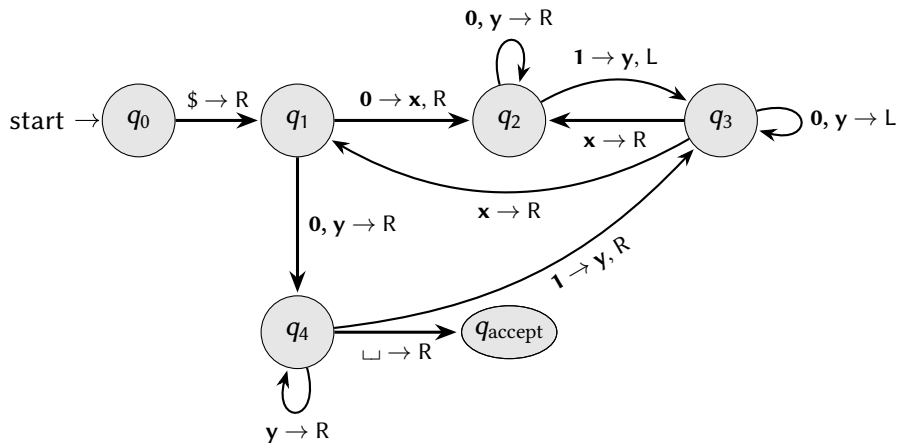
In each step we:

- Start at the left hand side and move right to the end of the string.
- When we see a state, we look at the next character and remember it using TM states.
- When we reach the end of the string we move left again until we reach the beginning of the string.
- When we see a state, we update it.

This has a time cost — linear programs in a multitape TM can become quadratic, and so on. But solvable problems are still solvable.

Non-deterministic Turing Machines (NTMs)

In non-deterministic TMs, there may be a choice of arrows out of a state:

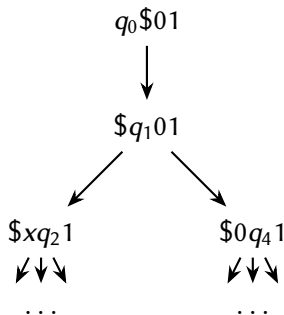


If *any* choice accepts, we accept.

Computation on NTMs

Configurations are more of a tree than a sequence of strings.

If we started with the input $\$01$, we'd get:



If there is a q_{accept} anywhere on this tree we accept.

Emulating NTMs

Idea: Do a BFS of the tree. Use three tapes:

- Configurations that we've seen but not searched.
- Current configuration.
- Configurations that we've searched.

We start here:

```
#q0$01␣  
␣  
␣
```

We move the first configuration from the first tape to the active tape:

```
xxxxx␣  
#q0$01␣  
␣
```

And we expand the current configuration:

```
xxxxx$q101␣  
␣  
#q0$01␣
```


Emulating NTMs (2)

If we repeat this a second time we get:

xxxxxxxxxx # xq_21 # $0q_41$ ␣
␣
q_001 # q_001 ␣

We keep running the search until we reach an *accept* state.

But time taken $\approx$ size of tree $\approx 2^{\mathcal{O}(\text{tree height})}$ — an exponential slowdown!

- *Huge* open question: can we do better?

TMs with RAM, and Others

Like TMs, but we can give the TM a memory address and let it jump to that position on the tape.

- *Once you have this, you basically have a CPU with memory — the details are different, but the behaviour, not so much.*

We won't cover this one in detail, but you can still emulate these TMs using regular TMs.

- *Again, there is a slowdown: linear algorithms can become n^6 algorithms.*

There are other ways of making TMs more powerful, but they all can be emulated by the regular TM (with slowdown), so long as:

- We allow arbitrary memory access, and
- Every step can only do a finite amount of work

The Church-Turing Thesis

What we believe — any feasible model of computation has equivalent power to a TM (with slowdowns):

The Church-Turing Thesis:

Anything that we can reasonably expect to decide with an algorithm can also be decided using a TM:

Problems that can be solved using algorithms $\Leftrightarrow$ Problems that can be solved using TMs

Universal TMs

One functionality that we expect programming languages to have: an EVAL function.

That is, we should be able to write programs that take code as input and then run said code.

So how to do this?

We'll build a TM **U** (a **universal TM**) that takes a TM M and a string w as input.

We'll use three tapes for **U**:

- 1 A tape that stores the transition functions and the string w ,
- 2 A tape that stores the tape of M , and
- 3 A tape that holds the current state of M .

Universal TMs (2)

We start by initializing tape 3 to q_0 and tape 2 to w .

We have an independent tape head on tape 2, and we'll use that to remember where we are on M 's tape.

To simulate a step of M :

- If M is in q_A , halt and accept, and if M is in q_R , halt and reject.
- Otherwise, simulate a step of M . We can read the third tape to see the current state of M , and we can use the second tape to find the current character in the tape.

We're skipping the details of how to encode an arbitrary Γ for M into $\mathbf{U}$, but you can see that in the long notes.

Language Classes

- We're not going to do a huge amount of work with actual TMs in this course, for the same reason that you don't usually do a lot of programming in assembly language.
- We really use them because they are a fairly simple model into which we can encode more complicated programs. If we can show that there's something that can't be done with TMs, we won't be able to do that thing with any computer program.
- We'll spend a lot of this course classifying different problems and languages depending on how well TMs are able to solve them.
- These classifications give us sets of languages, which we refer to as **language classes**.

Decidable Languages

- Our first language class is straightforward: if a language can be solved using TMs, it is **decidable**.

*If it cannot be solved using TMs, it is **undecidable**.*

- Here are a few examples:

PRIME: $\{ \langle n \rangle \mid n \in \mathbb{N} \text{ is prime} \}$

PATH: $\left\{ \langle G, s, t \rangle \mid \begin{array}{l} G \text{ is a directed graph with vertices } s \text{ and } t, \\ \text{and there is a path from } s \text{ to } t \end{array} \right\}$

SQUARE: $\{ \langle x, y \rangle \mid x, y \in \mathbb{N} \text{ and } y = x^2 \}$

Note that we haven't worried about the exact TMs for these languages — we know how to solve these problems, so there must be a TM for them!

- Notice that we've used set notation here to describe the languages:

$$\{ \langle \text{instance} \rangle \mid \langle \text{some decidable statement} \rangle \}$$

Remember this, we'll use it later.

Two Decidable Languages

Before we move on we'll introduce two very useful languages.

Suppose we define the following languages:

$$\begin{aligned} \text{ACC: } & \left\{ \langle M, w, k \rangle \mid \begin{array}{l} M \text{ is a TM, } w \text{ is its input, } k \text{ is a time limit, and} \\ M \text{ accepts } w \text{ in at most } k \text{ steps.} \end{array} \right\} \\ \text{H: } & \left\{ \langle M, w, k \rangle \mid \begin{array}{l} M \text{ is a TM, } w \text{ is its input, } k \text{ is a time limit, and} \\ M \text{ halts on input } w \text{ in at most } k \text{ steps.} \end{array} \right\} \end{aligned}$$

Since we have a universal TM **U** that takes an input $\langle M, w \rangle$ and runs M on w , these languages are *decidable*.

Two Undecidable Languages

But it turns out that not all languages are decidable. Here are two undecidable languages:

$A_{TM}: \{ \langle M, w \rangle \mid M \text{ is a TM, } w \text{ is its input, and } M \text{ accepts } w. \}$

$HALT: \{ \langle M, w \rangle \mid M \text{ is a TM, } w \text{ is its input, and } M \text{ halts on input } w. \}$

How do these languages relate to Acc and H?

Well, this time we haven't included a time limit. So we have to answer:

Does M accept w in one step?

Does M accept w in two steps?

Does M accept w in three steps?

...

Intuitively, if the program loops there's no stopping point for these questions.

That's not a proof, though. Let's have a try at proving this.

Liars and Undecidability

Our current proof will be based around the following idea:

A Liar's Paradox:

Suppose I were to say to you:

"The sentence that I am saying to you right now is false."

Am I telling the truth?

We're going to show that if A_{TM} were decidable we'd be able to encode this paradox in logic, which we can't reasonably expect to be able to do.

At a high level, we can see that the problem with the Liar's paradox is due to its self-reference. So we'll use A_{TM} to encode a similar self-reference.

How Can we Use This?

The overall proof is a proof by contradiction.

Basic Steps:

- Assume that A_{TM} is solvable. If so, there's a program (and, by the Church-Turing thesis, a TM) that solves it.
- Use that program to encode the Liar's paradox.
- Check that our encoding is not consistent and get a contradiction.
- Use contradiction to argue that our original assumption was wrong.

Let's follow the steps. . .

First: Assume A_{TM} is solvable.

- Then there is some TM M_A that decides it.
i.e., always halts, and always gives the right answer.
- We can call M_A as a function in other programs.

Programming a Liar's Paradox

Let's use M_A to program a Liar's paradox program D (D is for **diagonalization**).

- D will take some input w ,
- figure out what some TM (eventually D itself) would do on w , and
- do the opposite.

This will be easiest to do if w itself is a TM description $\langle M \rangle$. Then it can double as both input and as a reference to the TM that D emulates.

Here's the code:

$D =$ "On input $\langle M \rangle$:

- 1 Run M_A on the input $\langle M, M \rangle$.
- 2 If M_A accepts, *reject*.
- 3 Else, *accept*."

How Do We Get this Code?

The idea is that we want to use the assumption that M_A , a decider for A_{TM} , exists, and then use it to build a liar's paradox.

So the construction:

- $S = \text{"This sentence is false"}$
- If S is *true*, then it must be *false*.
- If S is *false*, then it must be *true*.

Becomes:

$D = \text{"On input } \langle w \rangle \text{:}$

- 1 Find out what D would do on (some) $\langle w \rangle$.
- 2 If it accepts, *reject*.
- 3 Else, *Accept*."

How Do We Get this Code? (2)

Now, if we're trying to program this code D , we can make design choices to make our job easier.

- Steps 2 and 3 are straightforward, so we don't have to worry too much about them. It's step 1 that's tricky.
- To run D inside the code in step 1, we need the code for D .
- This is easiest if we pass $\langle D \rangle$ in through the input.

So the code we write will be

$D =$ "On input $\langle M \rangle$:

We'll care about the $\langle M \rangle = \langle D \rangle$ case.

- 1 Find out what M would do on $\langle M \rangle$.
(We're running the input M in step 1 because we need the code, but we also run it on the input to D , as per the last slide.)
- 2 If it accepts, *reject*.
- 3 Else, *Accept*."

How Do We Get this Code? (3)

Finally, we have to figure out how to do the new step 1:

- 1 Find out what M would do on $\langle M \rangle$.

Fortunately, we have a tool for doing just this!

- Remember that we've assumed that a decider M_A for A_{TM} exists.
- Also remember that A_{TM} is precisely the set of TM description/input pairs $\langle M, w \rangle$ such that M accepts w .
- So if M accepts $\langle M \rangle$, then $\langle M, M \rangle \in A_{TM}$, and M_A will be able to tell us this!

So using M_A in this way, we get the code from the earlier slides:

$D =$ "On input $\langle M \rangle$:

- 1 Run M_A on the input $\langle M, M \rangle$.
- 2 If M_A accepts, *reject*.
- 3 Else, *accept*."

Getting a Contradiction

What happens if we run D on itself?

$D =$ “On input $\langle M \rangle$:

- ① Run M_A on the input $\langle M, M \rangle$.
 - ② If M_A accepts, *reject*.
 - ③ Else, *accept*.”
- If D (the emulated D) accepts, the the outer D rejects.
 - If D (the emulated D) rejects, the the outer D accepts.
 - D can't loop, since M_A has to halt.

So we have a contradiction, and this gives us our conclusion:

Since we can definitely build D if we have the TM M_A , and since we can't build D without getting a contradiction, it must not be possible to build M_A .

This means that A_{TM} is undecidable!

Decidability and Looping

Why did the D fail to work?

- Because the outer D emulates a second-level D which in turn emulates a third-level D which emulates a fourth-level D ...

If we try to actually build it, we get a loop, and there's never a good place to stop the loop!

If we allow looping programs, we can “solve” A_{TM} :

M_A = “On input $\langle M, w \rangle$:

- 1 Run M on w .
- 2 If it accepts, *accept*.
- 3 *Reject*.

This program accepts all *yes*-instances, and never accepts any *no*-instances. It can loop on *no*-instances, though.

Recognizability and Co-recognizability

The program

$M_A =$ “On input $\langle M, w \rangle$:

- 1 Run M on w .
- 2 If it accepts, *accept*.
- 3 *Reject*.

“recognizes” A_{TM} — if an input is in A_{TM} can eventually verify it for us.

- Problem: if the answer is no, there’s no point where we can stop and be sure of the answer.

So A_{TM} is, in a sense, *half-decidable*.

- If we can build a TM that recognizes the *yes*-instances of a language, like we can for A_{TM} , then we call the language **recognizable**.

*We also call the TM a **recognizer**.*

- There’s a dual concept, too: If $\bar{L}$ is recognizable, then L is **co-recognizable**.

Decidability, Recognizability, and Co-recognizability

How do these classes relate to each other?

- If we can decide a language (figure out all of its *yes* and *no*-instances), then we can figure out all of its *yes*-instances.
So every decidable language is recognizable.
- The example of A_{TM} shows us that the converse is not true, though.
- If we can decide a language (figure out all of its *yes* and *no*-instances), then we can figure out all of its *no*-instances.
So every decidable language is co-recognizable.
- The converse is also not true.

But, if a language L is *both* recognizable and co-recognizable, it must be decidable.

Dovetailing

Suppose some TM M_1 that recognizes L and another TM M_2 that recognizes $\bar{L}$. Then the following TM decides L :

M = “On input w :

- ① For $k = 0, 1, 2, \dots$:
- ② Run M_1 on w for k steps.
- ③ If it accepts, *accept*.
- ④ Run M_2 on w for k steps.
- ⑤ If it accepts, *reject*.

If $w \in L$, then M_1 eventually halts, and so does M . If $w \notin L$, then M_2 eventually halts, and so does M .

This type of process (the loop) is called **dovetailing**. Notice that we can't just run M_1 or M_2 on w without constraints, since they might loop. But limiting the number of steps k we take, and slowly increasing this limit, lets us get around this problem.

Effective Enumerations

Looping over numbers k is good, but dovetailing would be even more powerful if we could loop over strings and TMs as well.

We can loop over a set iff we can write a looping program that will print out the elements of that set.

For example, we can write a program that will print out

$$\epsilon, 0, 1, 00, 01, 10, 11, 000, 001, \dots$$

Every finite string in $\{0, 1\}^*$ will eventually be printed by this loop. This sort of loop is called an **effective enumeration**.

I've just showed you an effective enumeration of the strings (over $\{0, 1\}$), and I'll say without proof that there's an effective enumeration over the TM descriptions $\langle M \rangle$.

You can use these enumerations in your programs. If you want to enumerate other sets, you have to describe the enumeration.

Using Effective Enumerations

Consider the following program:

Let $w_0, w_1, \dots$ be an effective ordering of the strings,
and $\langle M_0 \rangle, \langle M_1 \rangle, \dots$ be an effective enumeration of the TMs.

$M =$ “Ignore input:

- ① For $k = 0, 1, 2, \dots$:
- ② For $i = 0$ to k :
- ③ For $j = 0$ to k :
- ④ Run M_i on w_j for k steps.
- ⑤ If it accepts, print $\langle M_i, w_j \rangle$ and continue.

We’ve used dovetailing and effective enumerations to print exactly the strings in A_{TM} .

This sort of program is called an **enumerator**.

- *A program is recognizable iff it has an enumerator.*

Dovetailing and effective enumerations are also very useful for building recognizers, too.

Another Undecidable Language

We don't want to go through diagonalization every time we get a computability result, so we'll need more tools.

Once we have one undecidable language, we can use that knowledge to find others, though:

Approach for showing a language L is undecidable:

- Assume that L is decidable. Then we can find a decider M_L for it.
- Use M_L to solve A_{TM} (or some other undecidable language)
- Conclude that L is not, in fact, solvable.

We'll develop this idea more next week, but here we'll use the basic idea to show that the *halting problem* is undecidable.

$$\text{HALT} = \{ \langle M, w \rangle \mid M \text{ halts on input } w \}$$

HALT is Undecidable

Suppose that we had a TM M_H that could decide HALT. Then consider:

$M' =$ “On input $\langle M, w \rangle$:

- ① Run M_H on $\langle M, w \rangle$.
- ② If it rejects:
- ③ *Reject*.
- ④ Else:
- ⑤ Run M on w .
- ⑥ If it accepts, *accept*, otherwise *reject*.

This program would decide A_{TM} .

Conclusion: HALT is not decidable!

- HALT is clearly recognizable, though — for the same reason as A_{TM} .
- Do you think that HALT is co-recognizable?

Next time: we'll strengthen this idea into something called a *mapping reduction*.